

ÉVALUATION EN COURS DE FORMATION

TP Développeur Web et Web Mobile

Copie à rendre.

Étudiant · Jassim Elghaouti

Formation · TP Développeur Web et Web Mobile (RNCP 37674)

Organisme · Studi — école 100% en ligne

Date · Mai 2026

Sujet · Application web pour « Vite & Gourmand », traiteur événementiel à Bordeaux

Client · Julie & José (FastDev en mission)

Introduction

Cette copie présente l'application web réalisée pour le client « Vite & Gourmand », traiteur événementiel à Bordeaux depuis vingt-cinq ans, dans le cadre de l'ECF du Titre Professionnel Développeur Web et Web Mobile.

L'application répond à l'intégralité des spécifications fonctionnelles de l'énoncé et est **déployée publiquement**. Elle expose les compétences des deux activités-types du référentiel : **front-end sécurisé** (AT1) et **back-end sécurisé** (AT2).

Liens utiles

Ressource	URL
Application déployée	https://vite-et-gourmand-drab.vercel.app
API backend	https://vite-et-gourmand-api-three.vercel.app/api
Code source GitHub (PUBLIC)	https://github.com/Jasteur91/vite-et-gourmand
Outil de gestion projet (Notion)	https://www.notion.so/3628e177f2a2815ab773e55b796f2602

Comptes de test fournis

Rôle	Email	Mot de passe
Administrateur	jose@vite-et-gourmand.fr	AdminViteG2026!
Employé	julie@vite-et-gourmand.fr	EmployeViteG2026!
Utilisateur	jean.dupont@example.com	DemoUser2026!

Cadre de cette copie. J'ai structuré ce document autour des 9 grandes questions implicites de l'énoncé. Pour chacune, je rappelle ce qui est demandé, puis j'explique mon approche et la justifie. Les détails techniques exhaustifs (MCD, diagrammes, déploiement pas-à-pas) sont dans les livrables annexes (*doc-technique.pdf*, *manuel-utilisateur.pdf*, *charte-graphique.pdf*, *gestion-projet.pdf*).

Comment ai-je interprété le besoin client et défini le périmètre fonctionnel ?

Le besoin du client est double : **augmenter la visibilité** (vitrine en ligne) et **industrialiser la prise de commande** (workflow). À partir des onze sections fonctionnelles décrites dans l'énoncé (page d'accueil, menu applicatif, pied de page, vue globale, création de compte, connexion, vue détaillée, commande, espaces utilisateur/employé/administrateur, contact), j'ai défini quatre rôles : *visiteur*, *utilisateur*, *employé*, *administrateur*, et un workflow de commande à huit statuts (*en_attente*, *accepte*, *en_preparation*, *en_cours_de_livraison*, *livre*, *en_attente_retour_materiel*, *terminee*, *annulee*).

J'ai veillé à respecter strictement les **règles métier exprimées** : minimum de personnes par menu, remise -10% au-delà du minimum +5, livraison gratuite Bordeaux / 5€ + 0,59€/km hors Bordeaux, prêt de matériel avec restitution sous 10 jours ouvrés (sinon 600€ de frais), création de compte administrateur impossible depuis l'application.

QUESTION 2 — CHOIX TECHNIQUES

Pourquoi ai-je retenu cette stack ?

L'énoncé n'impose aucune technologie, à l'exception d'une **base relationnelle** et d'une **base NoSQL**. Mon raisonnement pour chaque brique :

Frontend : React + Vite + TypeScript + Tailwind

React 18 est l'écosystème front le plus mature en 2026. Sa courbe d'apprentissage est progressive et la communauté garantit la disponibilité des solutions à n'importe quel problème. **Vite** remplace les anciens outils (CRA, Webpack) avec un dev server de ~50 ms de hot reload et un build Rollup ultra-optimisé. **TypeScript en mode strict** élimine une part importante des bugs en runtime (typo de champ, undefined, etc.). **Tailwind CSS** me permet de tenir une cohérence visuelle stricte via les design tokens (palette, typo, spacing) tout en gardant un bundle CSS final minuscule grâce à la purge automatique. **Framer Motion** apporte les animations subtiles attendues pour le style éditorial (parallax hero, apparitions au scroll, transitions hover).

Alternatives écartées : *Angular* trop opinionné pour un projet de cette taille ; *Vue* moins demandé sur le marché de l'emploi français ; *Next.js* overkill puisque nous n'avons besoin ni de SSR/SSG ni du file-system routing (le SEO du back-office n'a aucun intérêt).

Backend : Node.js + Express + TypeScript

Node 24 + Express 4 sont des piliers du back-end JavaScript et offrent un onboarding rapide pour tout développeur ou jury. Le code est **en TypeScript strict**, incluant `noUncheckedIndexedAccess` qui force la gestion des cas `undefined` sur les accès `req.user` ou `data?.field` — un filet de sécurité indispensable.

J'ai choisi **express-rate-limit + Helmet** pour la sécurité, **bcrypt** (cost 12, ~250 ms par hash) pour le hachage des mots de passe et **jsonwebtoken** (HS256) pour la session stateless. **Zod** est utilisée pour la

validation à la fois côté backend et frontend, ce qui me permet d'avoir un contrat unique sur la forme attendue des données (et la possibilité future de partager les schémas).

Alternatives écartées : *PHP + PDO* est l'exemple proposé par Studi mais nécessiterait un environnement runtime différent, avec moins d'élégance pour le partage de types front/back ; *NestJS* aurait été un overkill structurel pour ce périmètre.

Base de données relationnelle : PostgreSQL via Supabase

J'ai retenu **PostgreSQL 17** (région Paris) plutôt que MySQL pour la robustesse de ses contraintes (`CHECK` natives, types `ENUM` stricts, transactions ACID parfaites). **Supabase** est l'hébergement managé qui offre un free tier généreux (500 MB), des backups automatiques et une API REST auto-générée (utilisée par le client JavaScript que j'utilise depuis Express).

J'ai écrit **les scripts SQL natifs** moi-même (`database/migrations/001_initial_schema.sql` et `database/seed/001_demo_data.sql`) — conformément à l'exigence de l'énoncé. Aucune migration ORM n'est utilisée comme preuve de maîtrise.

Base de données NoSQL : MongoDB

L'énoncé impose une BDD non relationnelle pour le dashboard administrateur. J'ai choisi **MongoDB** : c'est le document store le plus enseigné dans les formations dev web (le jury Studi est familier), son **aggregation pipeline** est expressif (utilisé pour calculer en temps réel le nombre de commandes par menu et le CA), et le driver Node officiel est mature.

En développement, j'utilise `mongodb-memory-server` (Mongo en mémoire, téléchargé à la première run) ce qui supprime toute friction de setup. En production, c'est MongoDB Atlas (cluster M0 free, sur la même région que Supabase pour limiter la latence). MongoDB est connectée de manière **lazy** côté backend : si l'URI n'est pas configurée, les fonctions d'audit deviennent silencieusement no-op (le métier principal n'est pas bloqué).

Mailer : Resend

Pour les 7 templates de mail (bienvenue, confirmation commande, changement de statut, retour matériel, invitation avis, reset password, accusé contact), j'ai retenu **Resend**. C'est une API moderne, free tier 100 mails/jour, qui ne nécessite ni configuration DNS ni gestion de queue pour notre volume.

Déploiement : Vercel pour tout

Pour minimiser la complexité opérationnelle, j'ai déployé **les deux applications (front et back) sur Vercel**. Le frontend est un build Vite statique servi via le CDN global. Le backend Express est **wrappé en serverless function** via un handler unique (`api/index.ts`) qui exporte l'app Express. Cette approche m'a évité de découper l'API en multiples files-as-functions.

Comment ai-je conçu le modèle de données et pourquoi ces choix ?

Le MCD est conforme à l'annexe 1 de l'énoncé et a été **enrichi** pour gérer le workflow complet (reset password, tokens, contact, audit). Le schéma comporte 15 tables, toutes en 3NF.

Les choix structurants :

- **menu_plat en N:N** : l'énoncé précise qu'un plat peut être présent dans plusieurs menus. La table de jonction évite la duplication des plats.
- **menu_regime en N:N** : un menu peut être à la fois végétarien ET sans-gluten (la classification est flexible). L'énoncé invite d'ailleurs à « *alimenter d'avantage* » les régimes.
- **plat_allergene en N:N** : un plat peut contenir plusieurs allergènes, un allergène plusieurs plats.
- **Contraintes CHECK** sur les enums (statut, type de plat, jour de la semaine) — protection au niveau BDD complémentaire à la validation backend.
- **Triggers** `updated_at` automatiques sur les tables critiques (utilisateur, menu, commande).
- **Indexes** sur les colonnes filtrées (email pour le login, est_actif pour la liste publique, statut pour le filtre employé).
- **Table** `password_reset_token` avec expiration 1h et marquage `used_at` pour empêcher la réutilisation.

QUESTION 4 — SÉCURITÉ

Quelles dispositions ai-je prises sur la sécurité ?

Authentification

- **Hashage bcrypt cost 12** (~250 ms par hash) — résistant aux attaques par GPU/ASIC. Aucun mot de passe en clair n'existe en mémoire au-delà de la durée de la requête.
- **Politique de mot de passe** exhaustive : 10 caractères minimum, ≥1 majuscule, ≥1 minuscule, ≥1 chiffre, ≥1 caractère spécial. Indicateur visuel temps réel dans le formulaire d'inscription. Validation côté front ET back (regex partagée).
- **JWT signé HS256** avec secret 64 caractères. Expiration 7 jours. Stocké en `localStorage` côté navigateur. Choix justifié : pas de session côté serveur (stateless), pas de CSRF par défaut. Limite acceptée : impossible de révoquer un JWT avant expiration — pour une production critique, j'ajouterais un mécanisme de refresh token + blacklist Redis.
- **Reset password sécurisé** : endpoint `/auth/request-reset` répond toujours 200 (anti-enumeration). Token cryptographique 48 bytes (`crypto.randomBytes`), valide 1h, marqué `used_at` après usage.

Brute force et abus

- **express-rate-limit** : 10 tentatives / IP / 15 minutes sur `/api/auth/*` . Message HTTP 429 explicite.
- Rate-limit global : 120 requêtes / minute par IP sur toute l'API.

Élévation de privilège

- **RBAC** via middleware `requireRole('administrateur')`. Le rôle est dans le payload JWT, vérifié sur chaque requête.
- **Création d'admin impossible depuis l'API** — conformément à l'énoncé. José précise : « *vous devez lui créer ce compte et qu'il ne doit pas être possible de créer un compte Administrateur depuis l'application* ». Le compte admin est uniquement créé via le script SQL `seed/001_demo_data.sql`.
- Le rôle n'est **jamais** dans le body de l'API d'inscription (mass assignment impossible).

Protection des entrées

Risque OWASP	Mitigation
SQL Injection	Toutes les requêtes via le client Supabase (paramétrées). Aucune concaténation.
XSS	React échappe par défaut tout JSX. Pas de <code>dangerouslySetInnerHTML</code> . CSP via Helmet.
CSRF	API stateless avec JWT en Authorization header. Pas de cookie de session.
Mass assignment	Whitelist Zod sur chaque body. <code>role_id</code> jamais accepté.
Validation	Schémas Zod stricts (regex, length, enum).

En-têtes HTTP et CORS

Helmet active automatiquement : Content-Security-Policy, X-Frame-Options DENY, X-Content-Type-Options nosniff, Strict-Transport-Security, etc. CORS est configuré **strict** : seuls les sous-domaines `vite-et-gourmand*.vercel.app` et localhost sont autorisés.

QUESTION 5 — RGPD

Comment l'application respecte-t-elle le RGPD ?

- **Données minimales** : email, nom, prénom, téléphone, adresse, ville. Tout est nécessaire à la prestation (livraison + facturation + contact d'urgence). Pas de date de naissance ni de genre ou autre donnée non utile.
- **Consentement** à l'inscription (le formulaire mentionne explicitement l'acceptation des CGV et le traitement RGPD avec lien vers les mentions légales).
- **Droit d'accès et de rectification** : l'utilisateur peut modifier son profil depuis son espace.
- **Droit à l'effacement** : non automatisé dans l'interface (limitation assumée pour le périmètre ECF) mais documenté dans les mentions légales avec adresse email dédiée.
- **Pas de cookies de suivi tiers**. Le seul stockage navigateur est le JWT, à valeur fonctionnelle stricte — non soumis au consentement préalable.

- **Hébergement UE** : Supabase Paris (eu-west-3), Vercel a des edges UE pour les requêtes européennes.
- **Mots de passe hashés bcrypt** — jamais stockés en clair, jamais récupérables.

QUESTION 6 — ACCESSIBILITÉ

Comment l'application respecte-t-elle le RGAA ?

- **Sémantique HTML5** stricte (`header` , `nav` , `main` , `article` , `aside` , `footer`).
- **Contrastes WCAG AA** validés : cafe-900 (#2D1810) sur creme-200 (#F4ECDD) = ratio 12.4:1 ;
bordeaux-700 (#6B1F2A) sur creme-200 = ratio 8.1:1.
- **Navigation 100% clavier** — focus visible (`:focus-visible` avec ring bordeaux + offset).
- **Attributs alt** sur toutes les images.
- **Labels associés** à chaque `<input>` .
- **Pas d'auto-play**, pas de mouvement non contrôlable — la seule animation utilise `prefers-reduced-motion` (à enrichir).
- **Lecteur d'écran** : messages d'erreur Sonner ont des rôles ARIA implicites (aria-live polite).
- **Responsive** mobile-first jusqu'à 320px de large.

QUESTION 7 — MISE EN ŒUVRE DES FILTRES DYNAMIQUES

Comment ai-je implémenté les filtres "sans rechargement de page" ?

L'énoncé exige : « *il est nécessaire d'actualiser les menus affichés de manière dynamique (sans rechargement de page)* ».

Mon implémentation :

- Chaque filtre (prix min/max, nombre personnes, thème, régime) est un `useState` React.
- Un `useEffect` écoute les changements de filtres et déclenche un `fetch` vers `/api/menus?prix_max=...&theme=...` .
- Un `AbortController` annule la requête précédente si l'utilisateur change de filtre avant la fin de la réponse — évite les race conditions.
- L'API backend construit dynamiquement la requête Supabase en fonction des params (chaînage `.gte()` , `.lte()` , etc.).
- Le résultat met à jour l'état React → React re-render la grille avec une animation `AnimatePresence` de Framer Motion (apparition / disparition fluide).

QUESTION 8 — WORKFLOW DE COMMANDE

Comment ai-je géré le workflow complet (8 statuts, mails, audit) ?

Le workflow est documenté en backend dans `commande.routes.ts`. Le passage d'un statut à un autre est **guidé** côté frontend (l'employé ne voit que les transitions autorisées par le statut actuel).

À chaque changement de statut :

1. **Mise à jour PostgreSQL** : `UPDATE commande SET statut = X WHERE commande_id = Y`.
2. **Audit MongoDB** : insertion d'un document `order_events` avec `previous_statut`, `new_statut`, `actor_role`, `actor_id`, `metadata`. Ce log alimente :
 - l'historique visible dans l'espace utilisateur (suivi de commande détaillé) ;
 - les agrégations du dashboard administrateur (commandes par menu, CA).
3. **Email automatique** au client via Resend (template "status changed").
4. **Mail conditionnel** selon le nouveau statut :
 - `en_attente_retour_materiel` → mail dédié rappelant les 10 jours ouvrés et la facturation de 600€ ;
 - `terminee` → mail d'invitation à donner un avis.

L'utilisateur ne peut annuler que tant que le statut est `en_attente`. L'employé peut annuler à tout moment, à condition de saisir un motif et un mode de contact (GSM ou mail).

QUESTION 9 — DÉPLOIEMENT

Quelle est ma stratégie de déploiement et de mise en production ?

Le client exigeait explicitement le déploiement (« *des pénalités seront appliquées si l'application n'est pas en ligne* »). L'application est aujourd'hui déployée et opérationnelle 24/7 :

Couche	Hébergeur	URL
Frontend	Vercel (CDN)	vite-et-gourmand-drab.vercel.app
Backend	Vercel (serverless)	vite-et-gourmand-api-three.vercel.app/api
BDD relationnelle	Supabase (Paris)	uwzagqbyauuvivcvhxye.supabase.co
BDD NoSQL	MongoDB Atlas	(M0 free, prod)
Mailer	Resend	(API)

L'avantage de ce setup : déploiement **git push → auto-build → live en 90s**. Pas de Docker, pas de Kubernetes, pas de CI/CD à maintenir — le minimum viable. Les secrets sont gérés via le dashboard Vercel (chiffrés au repos).

Pour le gitflow, j'ai respecté la structure attendue :

- `main` : branche stable, reflet de la production.
- `develop` : branche d'intégration.
- `feature/database-schema` : branche de développement (qui contient le scaffolding initial complet — sur un projet d'équipe, j'aurais subdivisé en `feature/auth`, `feature/menus`, etc.).

Conclusion

L'application « **Vite & Gourmand** » répond à l'intégralité des fonctionnalités attendues par le client et démontre ma maîtrise des deux activités-types du référentiel TP-DWWM. Les choix techniques sont justifiés, les contraintes RGPD et RGAA sont prises en compte, et la sécurité (OWASP, hashage, rate limiting, JWT) est documentée et opérationnelle.

Le projet est livré **déployé, fonctionnel et testable** sans aucun setup préalable grâce aux comptes de démonstration. Le code source est public sur GitHub, les bonnes pratiques git sont respectées (branches, commits conventionnels). La documentation complète (technique, manuel utilisateur, gestion de projet, charte graphique) est fournie en annexe.

Pistes d'amélioration identifiées

Pour une montée en production réelle au-delà de l'ECF, j'identifie les évolutions suivantes :

- **Tests automatisés** : ajout de tests d'intégration Vitest sur les routes critiques et tests E2E Playwright pour les parcours utilisateur.
- **CI/CD** : pipeline GitHub Actions (lint + typecheck + tests + déploiement).
- **Monitoring** : intégration Sentry (front + back) pour le suivi des erreurs en production.
- **Païement en ligne** : intégration Stripe avec acompte à la commande.
- **PWA** : version mobile installable pour les usages des employés sur le terrain.
- **Refresh tokens** : système plus robuste pour la révocation des sessions JWT.
- **Calcul de distance réel** via l'API Google Maps pour le coût de livraison hors Bordeaux.